SkillNavora

STAGE 4 HANDBOOK

Stage 4: Application development

Outcome: Build complete AI-powered applications. Topics: FastAPI, React, Frontend and backend, Database integration, Authentication, Authorisation, SaaS fundamentals.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Stage 4: Application development

Outcome: Build complete AI-powered applications

This handbook covers each topic of Stage 4 with a definition, why it matters, how it works, step-by-step instructions, example code or templates, common mistakes, interview questions with answers, a hands-on exercise and official resources.

Topics in this stage

1. FastAPI
2. React
3. Frontend and backend
4. Database integration
5. Authentication
6. Authorisation
7. SaaS fundamentals

STAGE 4 · TOPIC 1

FastAPI

DEFINITION

> FastAPI is a modern Python web framework for building APIs. It uses Python type hints and Pydantic models to validate requests and responses automatically, supports async code, and generates interactive API documentation at /docs.

WHY IT MATTERS

> Most AI backends are Python services that expose model features as HTTP APIs to web apps, mobile apps and other systems. FastAPI is the most common choice for this because it is fast to write, fast to run and self-documenting.

HOW IT WORKS

- Create an app with app = FastAPI() and add routes with decorators (@app.get, @app.post).
- Path parameters (/leads/{lead_id}) and query parameters (?limit=10) come from function arguments.
- Request bodies are Pydantic models; invalid input returns a 422 error with details.
- Dependencies (Depends) inject shared things such as database sessions or the current user.
- async def routes let the server handle other requests while waiting for databases or AI APIs.
- Run with uvicorn (or the fastapi CLI); open /docs to test endpoints.

STEP-BY-STEP

1. pip install "fastapi[standard]".
2. Create main.py with the code below.
3. Run fastapi dev main.py and open http://127.0.0.1:8000/docs.
4. Create a lead with POST /leads and read it back with GET /leads/{id}.
5. Add a POST /leads/{id}/summary route that calls an LLM.
6. Write tests with FastAPI's TestClient and pytest.

EXAMPLE CODE / TEMPLATE

```
# pip install "fastapi[standard]" anthropic
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, EmailStr, Field
import anthropic

app = FastAPI(title='Lead API')
llm = anthropic.AsyncAnthropic()
LEADS: dict[int, dict] = {}

class LeadIn(BaseModel):
    name: str = Field(min_length=1)
    email: EmailStr
    message: str
    budget_lakh: float | None = None

@app.post('/leads', status_code=201)
async def create_lead(lead: LeadIn) -> dict:
    lead_id = len(LEADS) + 1
    LEADS[lead_id] = lead.model_dump()
    return {'id': lead_id, **LEADS[lead_id]}

@app.get('/leads/{lead_id}')
async def get_lead(lead_id: int) -> dict:
    if lead_id not in LEADS:
        raise HTTPException(status_code=404, detail='Lead not found')
    return LEADS[lead_id]

@app.post('/leads/{lead_id}/summary')
async def summarise(lead_id: int) -> dict:
    lead = await get_lead(lead_id)
    msg = await llm.messages.create(
        model='claude-opus-5-5', max_tokens=500,
        system='Summarise the enquiry in one sentence and suggest the next action.',
        messages=[{'role': 'user', 'content': lead['message']}])
    return {'summary': ''.join(b.text for b in msg.content if b.type == 'text')}

# Run: fastapi dev main.py   then open http://127.0.0.1:8000/docs
```

COMMON MISTAKES

- Blocking calls (requests, time.sleep) inside async routes; use async clients.
- No input validation, or returning internal error details to users.
- Secrets in code instead of environment variables.
- No tests for the main routes.

INTERVIEW QUESTIONS AND ANSWERS

Q. Why FastAPI for AI services?

A. Python ecosystem for AI, automatic validation with Pydantic, async support for slow I/O such as model calls, automatic OpenAPI docs, and high performance.

Q. What does Depends do?

A. It declares a dependency that FastAPI resolves for each request, such as a database session or the authenticated user, keeping routes small and testable.

Q. How would you handle a slow LLM call?

A. Use an async client with timeouts, stream the response if the user waits, or move long work to a background job and return a job ID the client can poll.

HANDS-ON EXERCISE

> Build a FastAPI service with endpoints to upload a document, ask a question about it, and list past questions, with tests.

OFFICIAL AND RECOMMENDED RESOURCES

- FastAPI: first steps (official) https://fastapi.tiangolo.com/tutorial/first-steps/
- FastAPI tutorial (official) https://fastapi.tiangolo.com/tutorial/
- Pydantic docs https://docs.pydantic.dev/latest/

React

DEFINITION

> React is a JavaScript library for building user interfaces from components: reusable functions that return markup (JSX) and update automatically when their data (state) changes. It is the most widely used front-end library and the base of Next.js and of most AI app builders' output.

WHY IT MATTERS

> Customers see the AI through an interface: chat windows, dashboards, review queues and forms. FDEs build or adjust these UIs, and code from Lovable, v0 and similar tools is React.

HOW IT WORKS

- Components: function ChatBox() { return ... }.
- Props pass data from parent to child; state (useState) holds data a component owns.
- Changing state re-renders the component with new data.
- Effects (useEffect) run side effects such as loading data after render.
- Events: onClick, onChange, onSubmit handlers update state.
- Lists render with .map and need a stable key; conditional rendering shows loading and error states.

STEP-BY-STEP

1. Create a project: npm create vite@latest ai-chat -- --template react-ts.
2. npm install and npm run dev; open the local URL.
3. Replace App.tsx with the chat component below.
4. Point it at your FastAPI endpoint (enable CORS on the backend).
5. Add loading, error and empty states.
6. Build for production with npm run build.

EXAMPLE CODE / TEMPLATE

```
// src/App.tsx
import { useState } from 'react';

type Msg = { role: 'user' | 'assistant'; text: string };

export default function App() {
  const [messages, setMessages] = useState<Msg[]>([]);
  const [input, setInput] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  async function send(e: React.FormEvent) {
    e.preventDefault();
    if (!input.trim()) return;
    const next = [...messages, { role: 'user' as const, text: input }];
    setMessages(next); setInput(''); setLoading(true); setError('');
    try {
      const res = await fetch('http://localhost:8000/chat', {
        method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ question: input }),
      });
      if (!res.ok) throw new Error(`Server error ${res.status}`);
```

```
      const data: { answer: string } = await res.json();
      setMessages([...next, { role: 'assistant', text: data.answer }]);
    } catch (err) {
      setError((err as Error).message);
    } finally {
      setLoading(false);
    }
  }

  return (
    <main style={{ maxWidth: 640, margin: '0 auto', padding: 16 }}>
      <h1>Property assistant</h1>
      {messages.length === 0 && <p>Ask about listings, prices or site visits.</p>}
      {messages.map((m, i) => <p key={i}><b>{m.role === 'user' ? 'You' : 'Assistant'}:</b> {m.text}
</p>)}
      {loading && <p>Thinking…</p>}
      {error && <p role="alert">Could not get an answer: {error}</p>}
      <form onSubmit={send}>
        <input value={input} onChange={e => setInput(e.target.value)} placeholder="Type your question"
/>
        <button disabled={loading}>Send</button>
      </form>
    </main>
  );
}
```

COMMON MISTAKES

- Mutating state directly instead of creating new arrays/objects.
- Missing keys in lists.
- Calling AI APIs with secret keys from the browser.
- No loading or error states.

INTERVIEW QUESTIONS AND ANSWERS

Q. What is state and why is it immutable?

A. State is data owned by a component that triggers re-renders when it changes. You replace it with new values (not mutate) so React can detect changes reliably.

Q. How should a React app call an LLM?

A. Through your own backend endpoint, which holds the API key, enforces auth and limits, and can stream the answer back.

HANDS-ON EXERCISE

> Build a React review queue that lists AI-drafted emails from your API, lets a user edit, approve or reject each, and shows status updates.

OFFICIAL AND RECOMMENDED RESOURCES

- React: learn (official) https://react.dev/learn
- Vite: getting started https://vite.dev/guide/
- TypeScript handbook https://www.typescriptlang.org/docs/handbook/intro.html

Frontend and backend

DEFINITION

> The frontend is the client-side interface users interact with (browser or app). The backend is the server-side system that holds business logic, data, secrets and integrations. They communicate through an API contract, usually REST with JSON over HTTPS.

WHY IT MATTERS

> A clean split keeps secrets and rules on the server, lets several clients (web, mobile, partners) share one backend, and lets teams work in parallel. Many security problems in AI apps come from putting backend responsibilities in the frontend.

HOW IT WORKS

- Frontend: renders UI, collects input, does light validation for usability, calls APIs, shows results.
- Backend: authenticates users, enforces permissions, validates all input again, talks to databases and AI providers, logs and monitors.
- API contract: endpoints, request and response shapes, error format; documented with OpenAPI.
- CORS: the browser only allows a page to call another origin if the backend permits that origin.
- Environments: local, staging and production each have their own URLs and keys.
- Streaming (server-sent events) lets the backend send AI tokens to the UI as they are generated.

STEP-BY-STEP

1. Write the API contract first (endpoints, JSON shapes, errors).
2. Build the backend endpoints with FastAPI and test them in /docs.
3. Enable CORS for the frontend's origin only.
4. Build the frontend against the contract.
5. Move all secrets to the backend's environment variables.
6. Deploy both and set the frontend's API base URL per environment.

EXAMPLE CODE / TEMPLATE

```
# Backend: allow only the frontend origin (FastAPI)
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:5173', 'https://app.example.com'],
    allow_methods=['GET', 'POST'],
    allow_headers=['Authorization', 'Content-Type'],
)

# Consistent error shape for every endpoint
from fastapi.responses import JSONResponse
@app.exception_handler(Exception)
async def unhandled(request, exc):
    # log exc with a request id here; never return internal details
    return JSONResponse(status_code=500, content={'error': 'internal_error', 'message': 'Something went
wrong'})

// Frontend: one API helper with the base URL from environment config
// const API = import.meta.env.VITE_API_URL;
```

```
// export async function api(path: string, body?: unknown) {
//   const res = await fetch(`${API}${path}`, { method: body ? 'POST' : 'GET',
//     headers: { 'Content-Type': 'application/json' }, body: body ? JSON.stringify(body) : undefined
});
//   if (!res.ok) throw new Error((await res.json()).message);
//   return res.json();
// }
```

COMMON MISTAKES

- Trusting frontend validation alone.
- allow_origins=['*'] with credentials in production.
- Leaking stack traces in API errors.
- Hard-coding production URLs in frontend code.

INTERVIEW QUESTIONS AND ANSWERS

Q. Why validate on both frontend and backend?

A. Frontend validation improves the user experience; backend validation is the real protection because any client (or attacker) can call the API directly.

Q. What is CORS?

A. A browser security mechanism that blocks a page from reading responses from another origin unless that server explicitly allows the page's origin.

HANDS-ON EXERCISE

> Write an OpenAPI contract for a document Q&A app, then build matching FastAPI and React parts and test them end to end.

OFFICIAL AND RECOMMENDED RESOURCES

- MDN: CORS https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS
- FastAPI: CORS https://fastapi.tiangolo.com/tutorial/cors/
- OpenAPI specification https://spec.openapis.org/oas/latest.html

Database integration

DEFINITION

Database integration connects your application code to a database to create, read, update and delete records safely and efficiently, usually through a driver plus an ORM (SQLAlchemy, SQLModel, Prisma) and a migration tool.

WHY IT MATTERS

Every real AI application stores users, conversations, documents, results, feedback and audit logs. Correct integration (schemas, migrations, transactions, connection pooling, parameterised queries) prevents data loss, leaks and slowdowns.

HOW IT WORKS

- A driver (psycopg for PostgreSQL) sends SQL over a connection.
- An ORM maps tables to Python classes and builds safe, parameterised SQL.
- Sessions group operations into transactions (commit or rollback).
- Connection pools reuse connections instead of opening one per request.
- Migrations (Alembic) version schema changes so every environment matches.
- Indexes and query design keep reads fast as data grows.

STEP-BY-STEP

1. Run PostgreSQL locally (Docker) or create a Supabase project.
2. pip install sqlmodel psycopg[binary].
3. Define models and create tables.
4. Add a FastAPI dependency that provides a session per request.
5. Write create and list endpoints.
6. Add Alembic for migrations before the first production release.

EXAMPLE CODE / TEMPLATE

```
# pip install sqlmodel "psycopg[binary]" "fastapi[standard]"
import os
from fastapi import FastAPI, Depends
from sqlmodel import SQLModel, Field, Session, create_engine, select

engine = create_engine(os.environ['DATABASE_URL'], pool_pre_ping=True)  #
postgresql+psycopg://user:pass@host/db

class Lead(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    org_id: int = Field(index=True)          # tenant isolation
    name: str
    email: str = Field(index=True)
    status: str = 'new'

SQLModel.metadata.create_all(engine)       # use Alembic migrations in production
app = FastAPI()

def get_session():
    with Session(engine) as session:
```

```
        yield session

@app.post('/orgs/{org_id}/leads')
def create(org_id: int, lead: Lead, s: Session = Depends(get_session)) -> Lead:
    lead.org_id = org_id
    s.add(lead); s.commit(); s.refresh(lead)
    return lead

@app.get('/orgs/{org_id}/leads')
def list_leads(org_id: int, s: Session = Depends(get_session)) -> list[Lead]:
    return list(s.exec(select(Lead).where(Lead.org_id == org_id).limit(100)))
```

COMMON MISTAKES

- Building SQL with f-strings and user input.
- Opening a new connection per query without pooling.
- Changing production schemas by hand without migrations.
- Forgetting the tenant filter on queries.

INTERVIEW QUESTIONS AND ANSWERS

Q. What is an ORM and its trade-offs?

A. It maps database tables to code objects, giving safer queries and less boilerplate, but can hide inefficient SQL; inspect generated queries and drop to SQL when needed.

Q. Why use migrations?

A. They record every schema change as versioned code so all environments are reproducible, changes can be reviewed and rolled back, and deployments are automated.

Q. What is the N+1 query problem?

A. Loading a list then running one extra query per item; fix it with joins or eager loading.

HANDS-ON EXERCISE

> Add a PostgreSQL database to your lead API with migrations, indexes, and a conversations table that stores every AI answer and user rating.

OFFICIAL AND RECOMMENDED RESOURCES

- SQLModel docs https://sqlmodel.tiangolo.com/
- SQLAlchemy docs https://docs.sqlalchemy.org/
- Alembic tutorial https://alembic.sqlalchemy.org/en/latest/tutorial.html
- Supabase database docs https://supabase.com/docs/guides/database/overview

Authentication

DEFINITION

> Authentication verifies who a user or system is: passwords, magic links, social or enterprise single sign-on (Google, Microsoft Entra ID, Okta), multi-factor authentication, and API keys or tokens for machines.

WHY IT MATTERS

> AI apps handle customer data and cost money per call. Without solid authentication anyone can read data or run up bills. Enterprise customers also require SSO and MFA before approving a deployment.

HOW IT WORKS

- The user proves identity to an identity provider (your auth service, Supabase Auth, Auth0, Entra ID, Okta).
- On success the app receives a session or a signed token (JWT) with the user ID, expiry and claims.
- Each API request carries the token (Authorization: Bearer ...) or a secure, HttpOnly session cookie.
- The backend verifies the signature and expiry before doing anything.
- OAuth 2.0 and OpenID Connect are the standards behind 'Sign in with Google/Microsoft' and enterprise SSO.
- Passwords are never stored in plain text; providers hash them with slow algorithms (bcrypt, Argon2).

STEP-BY-STEP

1. Use a managed provider instead of building auth yourself (Supabase Auth, Auth0, Clerk, Entra ID).
2. Add sign-up and login to the frontend with the provider's SDK.
3. Send the access token to your backend with each request.
4. In FastAPI, verify the token in a dependency and load the user.
5. Protect every non-public route with that dependency.
6. Turn on MFA and set short token lifetimes with refresh tokens.

EXAMPLE CODE / TEMPLATE

```
# Verify a Supabase (or other provider) JWT in FastAPI
# pip install pyjwt "fastapi[standard]"
import os, jwt
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

bearer = HTTPBearer()
SECRET = os.environ['JWT_SECRET']          # or verify with the provider's public keys (JWKS)

def current_user(creds: HTTPAuthorizationCredentials = Depends(bearer)) -> dict:
    try:
        claims = jwt.decode(creds.credentials, SECRET, algorithms=['HS256'], audience='authenticated')
    except jwt.ExpiredSignatureError:
        raise HTTPException(401, 'Session expired, please log in again')
    except jwt.InvalidTokenError:
        raise HTTPException(401, 'Invalid token')
    return {'id': claims['sub'], 'email': claims.get('email'), 'role': claims.get('role')}

@app.get('/me')
```

```
def me(user: dict = Depends(current_user)) -> dict:
    return user
```

COMMON MISTAKES

- Building your own password storage and login from scratch.
- Storing tokens in localStorage where scripts can steal them, without considering cookies.
- Long-lived tokens with no revocation.
- Checking login in the frontend only.

INTERVIEW QUESTIONS AND ANSWERS

Q. Authentication vs authorisation?

A. Authentication confirms identity (who you are); authorisation decides permissions (what you can do). Both run on every protected request.

Q. What is a JWT and what are its risks?

A. A signed token carrying claims such as user ID and expiry. Risks: it cannot easily be revoked before expiry, it must be verified properly (algorithm, signature, audience), and it must not carry secrets.

Q. What is SSO and why do enterprises require it?

A. Single sign-on lets employees use their company identity (via SAML or OpenID Connect) so IT controls access centrally, enforces MFA and removes access when people leave.

HANDS-ON EXERCISE

> Add Supabase Auth to your React + FastAPI app: login, protected routes, and a /me endpoint that verifies the token.

OFFICIAL AND RECOMMENDED RESOURCES

- OWASP Authentication Cheat Sheet https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Supabase Auth docs https://supabase.com/docs/guides/auth
- OAuth 2.0 simplified https://www.oauth.com/

Authorisation

DEFINITION

> Authorisation decides what an authenticated user or service is allowed to do and which data it may see: role-based access control (RBAC), attribute-based rules, resource ownership and tenant isolation, enforced in the backend and, ideally, in the database.

WHY IT MATTERS

> Most data leaks in multi-user apps are authorisation bugs: user A reads user B's record by changing an ID. In AI apps this extends to retrieval: RAG must only return documents the user may read, and agents must only call tools the user is allowed to use.

HOW IT WORKS

- Roles group permissions (owner, admin, agent, viewer).
- Every request checks the user's role and ownership of the resource (object-level authorisation).
- Tenant isolation: every query filters by the user's organisation.
- Row-level security (PostgreSQL/Supabase RLS) enforces rules inside the database as a second layer.
- AI-specific: filter retrieval by permission metadata; give agent tools the user's permissions, not admin rights.
- Log denied attempts for security monitoring.

STEP-BY-STEP

1. List roles and the actions each may perform (a permission matrix).
2. Add org_id and owner fields to every tenant table.
3. Write a FastAPI dependency require_role('admin').
4. Filter every query by org_id from the token, never from user input.
5. Enable RLS policies in the database.
6. Write tests that try to access another tenant's data and expect 403/404.

EXAMPLE CODE / TEMPLATE

```
from fastapi import Depends, HTTPException

PERMISSIONS = {
    'owner':  {'leads:read', 'leads:write', 'billing:manage', 'users:manage'},
    'admin':  {'leads:read', 'leads:write', 'users:manage'},
    'agent':  {'leads:read', 'leads:write'},
    'viewer': {'leads:read'},
}

def require(permission: str):
    def check(user: dict = Depends(current_user)) -> dict:
        if permission not in PERMISSIONS.get(user['role'], set()):
            raise HTTPException(403, 'You do not have permission for this action')
        return user
    return check

@app.delete('/leads/{lead_id}')
def delete_lead(lead_id: int, user: dict = Depends(require('leads:write')), s=Depends(get_session)):
    lead = s.get(Lead, lead_id)
```

```
    if not lead or lead.org_id != user['org_id']:   # object-level + tenant check
        raise HTTPException(404, 'Lead not found')
    s.delete(lead); s.commit()
    return {'deleted': lead_id}

-- Supabase / PostgreSQL row-level security as a second layer
-- alter table leads enable row level security;
-- create policy "members read own org" on leads for select
--   using (org_id in (select org_id from memberships where user_id = auth.uid()));
```

COMMON MISTAKES

- Checking the role but not resource ownership (IDOR vulnerability).
- Taking org_id from the request body instead of the verified token.
- Agents using an all-powerful service account.
- RAG search without permission filters.

INTERVIEW QUESTIONS AND ANSWERS

Q. What is an IDOR vulnerability?

A. Insecure direct object reference: the app trusts an ID from the user without checking ownership, so changing the ID exposes someone else's data. Fix with object-level checks and tenant filters.

Q. How do you apply authorisation to RAG?

A. Store access metadata (tenant, groups, document ACLs) with each chunk and filter retrieval by the requesting user's permissions before anything reaches the model.

Q. RBAC vs ABAC?

A. RBAC grants permissions by role; ABAC evaluates attributes of user, resource and context (department, region, time). Many systems combine them.

HANDS-ON EXERCISE

> Add roles and tenant isolation to your SaaS API, enable RLS, and write automated tests proving users cannot read or modify another organisation's data.

OFFICIAL AND RECOMMENDED RESOURCES

- OWASP Authorization Cheat Sheet https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- OWASP API Security Top 10 https://owasp.org/API-Security/
- Supabase: row level security https://supabase.com/docs/guides/database/postgres/row-level-security

STAGE 4 · TOPIC 7

SaaS fundamentals

DEFINITION

> Software as a service (SaaS) is software delivered online by subscription. A SaaS product needs more than features: multi-tenancy, sign-up and onboarding, organisations and roles, plans and billing, usage limits, admin tools, analytics, support and reliability.

WHY IT MATTERS

> Many FDE solutions become products, and AI agencies often grow into SaaS. Knowing the building blocks lets you design solutions that scale to many customers and earn recurring revenue.

HOW IT WORKS

- Multi-tenancy: one deployment serves many organisations; data is isolated by org_id (shared tables + RLS), separate schemas, or separate databases for large customers.
- Accounts: users belong to organisations through memberships with roles; invitations add teammates.
- Billing: plans (free, pro, enterprise) via Stripe or Razorpay; webhooks update the subscription status in your database.
- Metering: count usage (AI calls, documents, seats) per organisation and enforce plan limits.
- Onboarding: guide new organisations to first value quickly (sample data, checklists).
- Operations: admin console, product analytics, support, status page, backups and SLAs. Metrics: MRR, churn, activation, net revenue retention.

STEP-BY-STEP

1. Model the core tables: organisations, users, memberships, plans, subscriptions, usage_events.
2. Add sign-up that creates an organisation and an owner membership.
3. Integrate Stripe Checkout (or Razorpay) and handle subscription webhooks.
4. Record usage per AI call and block calls above the plan limit.
5. Build an admin page listing organisations, plans and usage.
6. Track activation events (first document uploaded, first answer accepted).

EXAMPLE CODE / TEMPLATE

```
-- Core SaaS tables (PostgreSQL)
create table organisations (id uuid primary key default gen_random_uuid(), name text not null, plan
text not null default 'free', created_at timestamptz default now());
create table memberships (org_id uuid references organisations(id), user_id uuid not null, role text
not null check (role in ('owner','admin','member','viewer')), primary key (org_id, user_id));
create table usage_events (id bigserial primary key, org_id uuid references organisations(id), kind
text not null, units int not null default 1, at timestamptz default now());
create index on usage_events (org_id, at);

# Enforce plan limits before an AI call (Python)
LIMITS = {'free': 100, 'pro': 5000, 'enterprise': None}   # AI calls per month

def check_quota(s, org) -> None:
    limit = LIMITS[org.plan]
    if limit is None:
        return
    used = s.exec(text("select coalesce(sum(units),0) from usage_events where org_id=:o and
kind='ai_call' and at ≥ date_trunc('month', now())"), {'o': org.id}).one()[0]
```

```
    if used ⩾ limit:
        raise HTTPException(402, f'Monthly limit of {limit} AI calls reached. Upgrade your plan to
continue.')
```

COMMON MISTAKES

- Adding multi-tenancy late, after data is mixed.
- Trusting the frontend for plan status instead of billing webhooks.
- No usage metering, so AI costs exceed revenue.
- No admin tools, so support needs database access.

INTERVIEW QUESTIONS AND ANSWERS

Q. How do you price an AI SaaS product?

A. Start from customer value and your cost per unit (tokens, compute); combine a subscription with included usage and overage or tiers; keep gross margin healthy and test pricing with real customers.

Q. Single-tenant vs multi-tenant?

A. Multi-tenant shares infrastructure for lower cost and easier updates but needs strict isolation; single-tenant gives each customer separate infrastructure for stronger isolation and customisation at higher cost, often for large enterprises.

Q. Which SaaS metrics matter early?

A. Activation rate, weekly active organisations, retention cohorts, churn, MRR and gross margin (including AI costs).

HANDS-ON EXERCISE

> Turn your document Q&A app into a multi-tenant SaaS with organisations, roles, a free and a pro plan, Stripe test-mode billing and usage limits.

OFFICIAL AND RECOMMENDED RESOURCES

- Stripe billing docs https://docs.stripe.com/billing
- Supabase docs https://supabase.com/docs
- Y Combinator Startup School https://www.startupschool.org/
- AWS: SaaS lens (multi-tenancy) https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/saas-lens.html